

1장 실험

시스템 콜로 보는 운영체제 · MIT 6.1810 강의 1과 교재 1장을 xv6에서 직접 돌려 보며

MIT 6.1810(2026 가을)의 첫 강의는 운영체제를 “응용 프로그램 아래, 하드웨어 위에 있는 층”으로 소개하고, 그 층과 대화하는 유일한 통로인 시스템 콜을 예제 아홉 개로 보여 준다. 이 문서는 그 예제를 xv6에서 하나씩 실제로 돌려 보고, 출력이 왜 그렇게 나오는지를 xv6 소스에서 확인한 기록이다. 강의 노트만 읽을 때는 지나치기 쉬운 현상, 예를 들어 출력이 글자 단위로 뒤섞이거나 `ls > out`이 수십 배 느린 이유도 같이 따라갔다.

2026년 10월 4--7일 기록. 대상: xv6-riscv 06aad25 + 강의 예제(lec1 브랜치), 교재 2026-09-23판. 화면 그림 중 “실제 화면”은 작업 중 찍은 스크린샷이고, “tmux 캡처”는 xv6를 tmux 창에서 실제로 돌린 뒤 `tmux capture-pane`으로 받은 글자를 창 모양으로 그린 것이다. 글자는 고치지 않았다. 밑줄 친 쪽 번호를 누르면 교재 PDF의 그 쪽이 열린다.

1. 범위: 강의 1은 교재의 어디인가

강의 1은 교재 1장 Operating system interfaces (9–20쪽) 전체에 해당한다. 강의의 예제 순서와 교재의 절 순서가 거의 같아서, 아래 표대로 나란히 읽으면 된다.

강의 1 주제	예제	교재	함께 볼 코드
OS의 목표, 사용자/커널	–	1장 도입 (9–10쪽), 그림 1.1	–
<code>read, write</code>	ex1	1.2 (13쪽)	<code>user/usys.S</code>
<code>open</code> , FD 번호	ex2	1.2 (13–14쪽)	<code>user/init.c</code> , <code>kernel/sysfile.c</code>
<code>fork</code>	ex3	1.1 (10–11쪽)	<code>kernel/proc.c:kfork</code>
<code>exec, wait</code>	ex4, ex5	1.1 (11–13쪽)	<code>kernel/exec.c:kexec</code>
리다이렉션	ex6	1.2 (14–16쪽)	<code>user/sh.c:REDIR</code>
파이프	ex7, ex8	1.3 (16–17쪽)	<code>user/sh.c:PIPE</code> , <code>kernel/pipe.c</code>
디렉터리 읽기	ex9	1.4 (17–19쪽)	<code>kernel/fs.h</code>
시스템 콜 목록	–	그림 1.2 (11쪽)	–
설계 이야기	–	1.5 Real world (19쪽)	–
연습 문제	–	1.6 Exercises (20쪽)	–

실험 도중 강의 1의 범위를 넘어 미리 들여다본 곳도 있다. 나중에 해당 강의에서 다시 만난다.

- 시스템 콜이 커널로 들어가는 경로: 2.2절(22쪽), 4.3절(47쪽) — 강의 6
- 콘솔 입력 처리: 6.1절 Code: Console input(57쪽) — 강의 11

2. 실습 환경과 전체 구조

2.1 무엇이 무엇 위에서 도는가

xv6는 실제 RISC-V 보드가 아니라 노트북 위의 QEMU가 흉내 내는 가상 기계에서 돈다. 이 문서에서 “커널”, “장치”, “사용자 프로그램”이라고 부르는 것이 그림의 어느 층인지 먼저 잡아 두면 이후 설명을 따라가기 쉽다.

사용자 공간 U-mode	init	sh	ex1 ... ex9		cat, ls, echo ...	
시스템 콜 경계 · ecall로 들어가고 sret으로 돌아온다						
xv6 커널 S-mode	프로세스 proc.c	메모리 vm.c	파일 시스템 fs.c log.c	파이프 pipe.c	콘솔 드라이버 console.c	디스크 드라이버 virtio_disk.c
가상 기계 QEMU virt	CPU 3개 hart 0, 1, 2	RAM 128MB	타이머 · 인터럽트 컨트롤러 CLINT, PLIC		UART 16550 콘솔 장치	virtio-blk 디스크 장치
호스트 Ubuntu 22.04	빌드: riscv64-unknown-elf-gcc, mkfs → kernel/kernel, fs.img				터미널 Ghostty + tmux	디스크 이미지 fs.img

그림 1. 실습 환경의 층. 아래에서 위로 쌓인다. 커널의 콘솔·디스크 드라이버는 바로 아래 칸의 UART·virtio 장치를 다룬다.

부팅 순서는 `kernel/entry.S` → `start.c`(M-mode) → `main.c`에서 각 부분을 준비하고, 첫 사용자 프로세스 `init`을 만든다. `init`이 콘솔을 열고 셸 `sh`를 띄우면 우리가 보는 `$` 프롬프트가 나온다. 이 문서의 예제는 모두 그 셸이 `fork`와 `exec`으로 실행한 사용자 프로그램이다.

2.2 예제 넣기

강의 노트의 예제(`ex1.c`~`ex9.c`)를 xv6 원본 저장소의 `user/`에 넣고, `Makefile`의 `UPROGS`에 `$U/_ex1` 같은 줄을 더해 디스크 이미지에 들어가게 했다.

```
$ cd ~/xv6-riscv && git switch lec1
$ make qemu
```

```

xv6 · qemu-system-riscv64

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls
.          1 1 1024
..         1 1 1024
README    2 2 2441
...
ex1        2 23 35696
ex2        2 24 35584
ex3        2 25 35544
ex4        2 26 35488
ex5        2 27 35912
ex6        2 28 35936
ex7        2 29 35680
ex8        2 30 35856
ex9        2 31 35832
console    3 32 0
$ 

```

tmux 캡처. 부팅 직후 `ls`. 디스크 이미지에 `ex1--ex9`가 들어 있다.

3. 시스템 콜이라는 경계

xv6에서 사용자 프로그램은 장치나 다른 프로세스의 메모리를 직접 건드릴 수 없다. 할 수 있는 일은 커널에 부탁하는 것뿐이고, 그 부탁이 시스템 콜이다. C 코드에서는 `read(0, buf, 64)`처럼 평범한 함수 호출로 보이지만, 실제로 링크되는 `read`는 `user/usys.S`에 있는 짧은 어셈블리다.

```

# user/usys.S (generated by usys.pl)
read:
    li a7, SYS_read      # a7 = system call number
    ecall                 # trap into the kernel (U-mode -> S-mode)
    ret

```

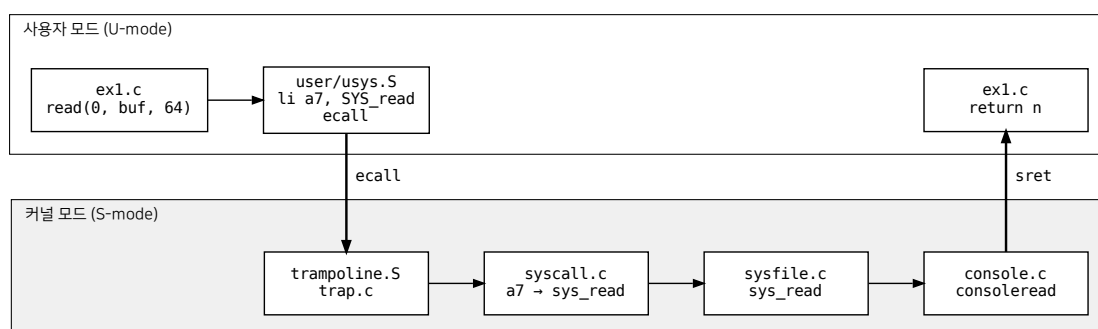


그림 2. `read(0, buf, 64)` 한 번이 지나가는 길. 사용자 모드에서 `ecall`로 들어가 커널을 거쳐 `sret`으로 돌아온다. 가운데 단계(트랩핑, 트랩 처리)는 강의 6에서 다룬다.

`ecall` 한 줄이 CPU의 권한 수준을 올리고 커널의 정해진 입구로 뛰어든다. 커널은 `a7`을 보고 `sys_read(kernel/sysfile.c:69)`를 부른 뒤, 일을 마치면 권한을 다시 낮춰 `ret` 다음으로 돌아온다. 지금은 함수처럼 보이지만 경계를 넘는다는 것만 기억하면 된다.

핵심 사용자 프로그램이 커널에 일을 시키는 방법은 시스템 콜뿐이다. C에서는 함수 호출로 보이지만, 실제로는 `ecall` 명령어로 U-mode에서 S-mode로 넘어간다.

4. ex1: 읽고 그대로 쓰기

4.1 원본

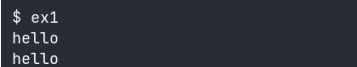
```
int
main()
{
    char buf[64];

    while(1){
        int n = read(0, buf, sizeof(buf));
        if(n <= 0)
            break;
        write(1, buf, n);
    }

    exit(0);
}
```

첫 번째 인자 0과 1은 파일 디스크립터(FD)다. 관례상 0은 표준 입력, 1은 표준 출력이다. read는 요청한 64 바이트보다 적게 읽을 수 있으므로, 쓸 때는 sizeof(buf)가 아니라 실제로 읽은 n을 넘긴다. 0이 돌아오면 입력 끝이다.

4.2 실행



```
$ ex1
hello
hello
```

실제 화면. hello를 치고 Enter를 누르면 두 번 보인다.

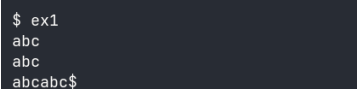
첫 줄은 내가 친 글자를 커널이 화면에 되돌려 찍어 준 것(에코)이고, 둘째 줄이 ex1이 read로 받아write한 것이다. 코드는 그대로 두고 입력만 바꿔 봤다.

```
$ echo hello | ex1
hello
$ ex1 < README | wc
48 336 2441
```

wc가 센 2441바이트는 호스트에서 켜진 README의 크기(wc -c README → 2441)와 같다. ex1은 FD 0 뒤에 키보드가 있는지, 파이프가 있는지, 파일이 있는지 모른다. 그래도 똑같이 동작한다. 누가 FD 0을 바꿔 끼우는지는 7절에서 확인한다.

4.3 왜 Enter를 쳐야 출력되는가

ex1을 띄우고 글자를 치는 동안에는 아무것도 출력되지 않다가, Enter를 치는 순간 한 줄이 나온다. ex1에는 줄 단위로 모으는 코드가 없으니, 모으는 쪽은 커널이다. 두 번째 줄에서 Enter 대신 Ctrl-d를 눌러 확인했다.

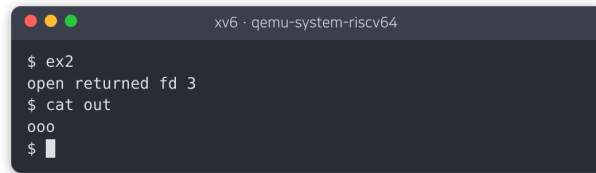


```
$ ex1
abc
abc
abccabc$
```

실제 화면. 첫 abc는 Enter, 두 번째 abc는 Ctrl-d로 끝냈다. Enter 없이도 바로 abc가 한 번 더 찍히고(abccabc), 한 번 더 누른 Ctrl-d에 ex1이 끝나 \$가 붙었다.

5. ex2: open과 FD 번호

```
int fd = open("out", O_WRONLY | O_CREATE | O_TRUNC);
printf("open returned fd %d\n", fd);
write(fd, "ooo\n", 4);
```



```
xv6 - qemu-system-riscv64
$ ex2
open returned fd 3
$ cat out
ooo
$
```

tmux 캡처. 처음 연 파일인데 FD가 3이다.

0, 1, 2는 이미 차 있기 때문이다. 첫 사용자 프로세스 `init`이 콘솔을 열고 두 번 복제해 두었고(`user/init.c:19--24`), 그 뒤의 모든 프로세스는 `fork`로 이 세 개를 물려받는다.

```
if (open("console", O_RDWR) < 0) { // -> fd 0
    mknod("console", CONSOLE, 0);
    open("console", O_RDWR);
}
dup(0); // stdout // -> fd 1
dup(0); // stderr // -> fd 2
```

5.1 FD는 커널 안 표의 칸 번호다

FD가 실제로 가리키는 것을 커널에서 따라가면 두 단계 구조가 나온다. 프로세스마다 있는 배열 `ofile[]`(`kernel/proc.h:101`, 16칸)과, 시스템 전체가 함께 쓰는 `struct file`(`kernel/file.h:1`, 최대 100개)이다.

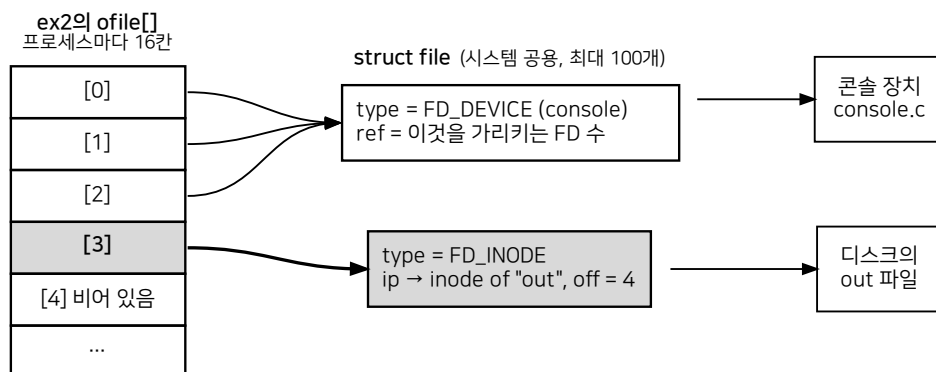


그림 5. `ex2`가 `open`한 직후의 커널 자료구조. FD는 `ofile[]`의 칸 번호일 뿐이고, 열린 파일의 실제(종류, 읽기/쓰기 위치, 참조 수)는 `struct file`에 있다. 파일 이름은 `open`할 때만 쓰인다.

새 FD를 고르는 곳은 `kernel/sysfile.c:40`의 `fdalloc`이다.

```
for (fd = 0; fd < NOFILE; fd++) {
    if (p->ofile[fd] == 0) {
        p->ofile[fd] = f;
        return fd;
    }
}
```

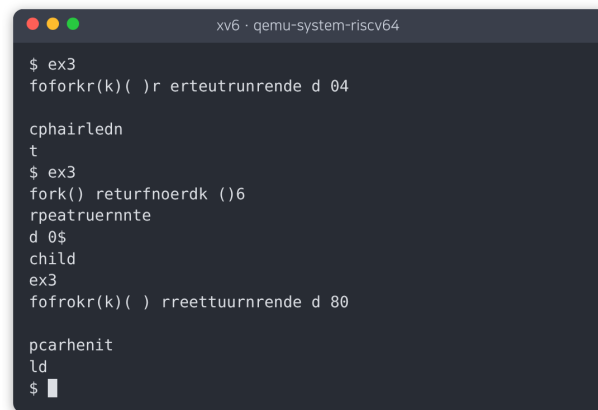
0번부터 훑어 비어 있는 가장 작은 번호를 준다. 교재 14쪽의 “the lowest-numbered unused descriptor”가 이 반복문이다. 이 규칙이 7절 리다이렉션의 열쇠가 된다.

핵심 FD는 프로세스마다 있는 배열 `ofile[]`의 인덱스다. 새 FD는 비어 있는 가장 작은 인덱스로 정해진다.

6. ex3--ex5: fork, exec, wait

6.1 fork: 출력이 뒤섞인다

```
pid = fork();
printf("fork() returned %d\n", pid);
if(pid == 0) printf("child\n");
else        printf("parent\n");
```



```
$ ex3
foforkr(k)( )r erteutrunrende d 04

cphairledn
t
$ ex3
fork() returfnorerk ()6
rpeatruernnte
d 0$
child
ex3
fofrokr(k)( ) rreettuurnrende d 80

pcarhenit
ld
$
```

tmux 캡처. 같은 프로그램을 세 번 실행했는데 세 번 모두 다르게 섞였다.

첫 번째 실행의 `foforkr(k)( )r ...`는 부모와 자식의 `fork()` returned가 글자 단위로 섞인 것이다. 이유는 xv6의 `printf`에 있다. `user/printf.c:10`의 `putc`가 한 글자마다 `write`를 한 번씩 부른다.

```
static void
putc(int fd, char c)
{
    write(fd, &c, 1);
}
```

부모와 자식은 CPU 세 개(-smp 3) 위에서 동시에 돌고, 각자 한 글자짜리 `write`를 연달아 부르니 커널에 도착하는 순서대로 섞인다. 두 번째 실행에서는 `$가 d 0` 뒤에 붙었다. 부모가 먼저 끝나서 쉼의 `wait`가 돌아오고 쉼이 프롬프트를 찍었는데, 그때 자식은 아직 출력 중이었다는 뜻이다. 그 바람에 세 번째 `ex3`은 프롬프트 없이 입력된 것처럼 보인다.

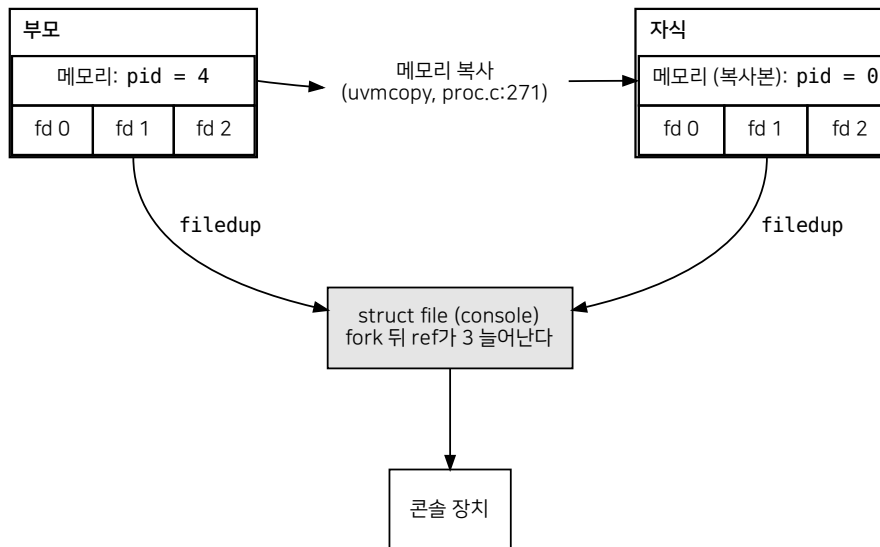


그림 6. `fork` 직후. 메모리는 복사되어 서로 독립이고, 열린 파일은 같은 `struct file`을 공유한다(`filedup`이 `ref`만 늘린다). 그래서 둘 다 같은 콘솔에 쓰고, 출력이 섞일 수 있다.

`kernel/proc.c:259`의 `kfork`에서 볼 곳은 세 줄이다.

```
:271  uvmcopy(p->pagetable, np->pagetable, p->sz)    // copy user memory
:282  np->trapframe->a0 = 0;                          // child's fork() returns 0
:287  np->ofile[i] = filedup(p->ofile[i]);           // share open files
```

자식의 반환값이 0인 이유는 자식이 처음 사용자 공간으로 돌아갈 때 `a0` 레지스터, 곧 함수 반환값 자리에 0을 넣어 두었기 때문이다.

6.2 `exec`: 돌아오지 않는 호출

```
char *argv[] = { "echo", "this", "is", "echo", 0 };
exec("echo", argv);
printf("exec failed!\n");
```

`exec failed!`는 찍히지 않는다. `exec`이 성공하면 그 아래 코드는 이미 메모리에서 사라졌기 때문이다. `kernel/exec.c:132`의 “Commit to the user image”에서 새 페이지 테이블로 갈아 끼우고 옛것을 버린다. 반면 `ofile[]`은 건드리지 않는다. `exec`은 메모리를 바꾸고 FD는 그대로 둔다. `ex4 | wc`가 동작하는 것도 그래서다.

6.3 `fork + exec + wait` = 셸이 명령 하나를 실행하는 법

```
xv6 · qemu-system-riscv64
$ ex4
this is echo
$ ex4 | wc
1 3 13
$ ex5
parent waiting
THIS IS ECHO
the child exited with status 0
$
```

tmux 캡처. `ex4`, `ex4 | wc`, `ex5`. 이번 실행에서는 `ex5`의 출력이 섞이지 않았다.

```

xv6 · qemu-system-riscv64

$ ex5
parent waitinTHIS gIS
ECHO
the child exited with status 0
$

```

앞선 실행(runs/run1.txt)의 ex5. 부모의 `parent waiting`과 자식이 `exec`한 `echo`의 출력이 섞였다.

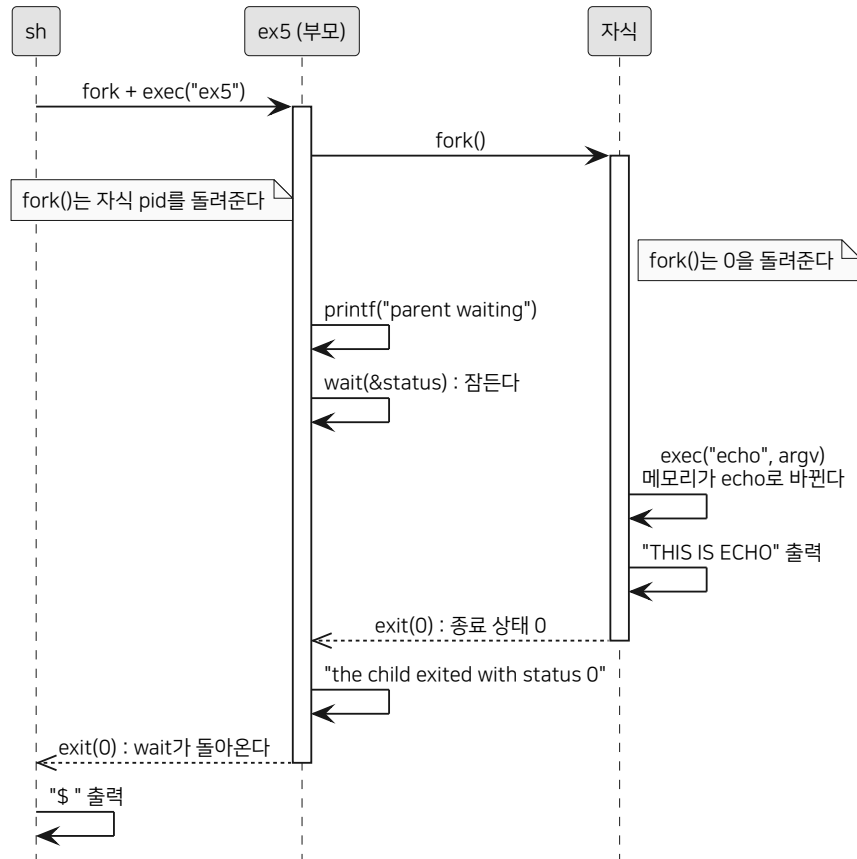


그림 7. 셸이 `ex5`를 실행하고, `ex5`가 다시 `echo`를 실행하는 흐름. 세로가 시간이다. 셸도 명령마다 똑같이 `fork` → `exec` → `wait`를 한다.

핵심 `fork`는 메모리를 복사하고 열린 파일은 공유한다. `exec`은 메모리만 새 프로그램으로 바꾸고 FD는 그대로 둔다. `wait`는 자식의 종료 상태를 받는다.

7. ex6: 리다이렉션은 셸이 한다

```

if(pid == 0){
    close(1);
    open("out", O_WRONLY | O_CREATE | O_TRUNC); // gets fd 1
    char *argv[] = { "echo", "ex6's", "redirected", "echo", 0 };
    exec("echo", argv);
}

```

```
xv6 · qemu-system-riscv64
$ ex6
$ cat out
ex6's redirected echo
$
```

tmux 캡처. ex6은 화면에 아무것도 찍지 않고, echo의 출력은 out 파일로 갔다.

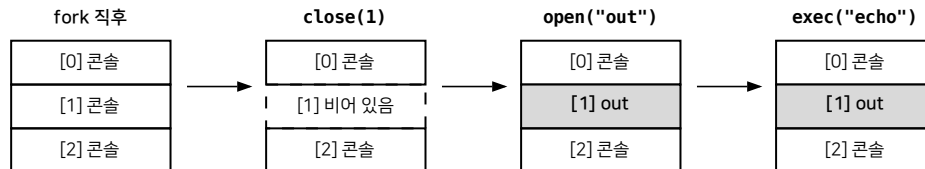


그림 8. ex6의 자식에서 ofile[]이 바뀌는 순서. close(1)이 비운 칸을 다음 open이 가져간다. exec은 이 표를 그대로 두므로 echo는 모른 채 파일에 쓴다.

셸의 <, > 처리도 똑같다(user/sh.c:83).

```
case REDIR:
    rcmd = (struct redircmd *)cmd;
    close(rcmd->fd); // 0 for '<', 1 for '>'
    if (open(rcmd->file, rcmd->mode) < 0) { // takes the freed slot
        fprintf(2, "open %s failed\n", rcmd->file);
        exit(1);
    }
    runcmd(rcmd->cmd); // then exec the command
```

open의 반환값을 저장조차 하지 않는다. 방금 비운 칸이 가장 작은 빈 칸이라는 것을 알기 때문이다. 실험 중 글자 하나가 빠져 들어갔을 때 본 메시지도 ex1이 아니라 여기서 셸이 낸 것이었다.

```
$ ex1
hello
hello
$ ex1
README
README
ex1 < README
ex1 < README
$ ex1 < README
open READM failed
$
```

실제 화면. 위쪽 네 줄은 ex1이 아직 실행 중일 때 친 글자라서 그대로 되돌아 나왔다. Ctrl-d로 빠져나온 뒤 입력한 ex1 < README는 끝 글자가 전달되지 않아 셸의 open이 실패했다. ex1.c에는 open이 없다.

fork와 exec이 따로 있는 이유가 여기서 드러난다. 둘 사이의 틈에서 자식은 자기 FD를 마음대로 바꿀 수 있고, 부모(셸)의 FD는 건드리지 않는다(교재 15쪽).

7.1 덤: ls > out은 왜 느린가

리다이렉션을 시험하다 ls > out이 유난히 오래 걸리는 것을 발견했다. 같은 명령을 콘솔로 보낼 때와 시간을 비교했다. (이 노트북의 QEMU에서 잦 값이라 환경마다 다르다.)

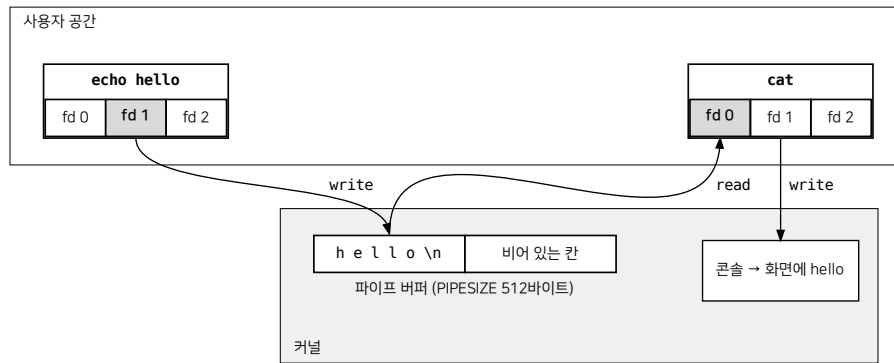


그림 10. `echo hello | cat`이 조립된 뒤의 모습. 두 프로그램은 평소처럼 FD 1에 쓰고 FD 0에서 읽을 뿐이고, 둘을 커널의 파이프 버퍼로 이은 것은 셸이다. 셸 자신은 파이프 끝을 모두 닫고 `wait`한다.

여기서도 `close` 후 `dup`이 가장 작은 빈 칸을 받는 규칙을 쓴다. 쓰고 남은 파이프 끝을 모두 닫는 것도 중요하다. 쓰는 쪽 FD가 하나라도 열려 있으면 읽는 쪽은 입력 끝(0바이트)을 영영 받지 못한다(교재 16-17쪽).

핵심 파이프는 커널 버퍼 하나에 읽기 FD와 쓰기 FD를 붙인 것이다. 쓰기 FD가 모두 닫혀야 읽는 쪽이 입력 끝(0)을 받는다.

9. ex9: 디렉터리도 읽을 수 있는 파일이다

```
struct dirent { ushort inum; char name[14]; };
fd = open(".", 0);
while(read(fd, &e, sizeof(e)) == sizeof(e))
    if(e.name[0] != '\0') printf("%s\n", e.name);
```



tmux 캡처. 루트 디렉터리의 항목 이름만 나온다(가운데 생략).

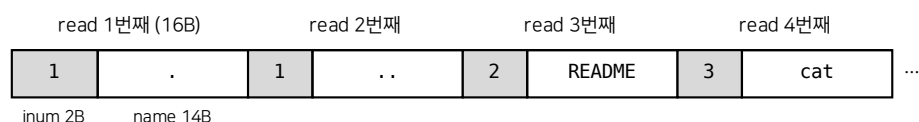


그림 11. 디렉터리 파일의 내용. 16바이트짜리 `struct dirent(kernel/fs.h:58)`가 이어 붙어 있고, `read(fd, &e, 16)` 한 번이 한 칸을 읽는다. 이름은 최대 `DIRSIZ = 14`바이트다.

`ls`는 여기에 `stat`으로 종류와 크기를 덧붙여 보여 줄 뿐이다(`user/ls.c`).

핵심 디렉터리도 `open`과 `read`로 읽는다. 내용은 16바이트 `struct dirent`의 배열이다.

10. 정리

1. 운영체제의 목표는 다중화, 격리, 공유, 추상화다. 그 경계에 시스템 콜이 있다.
2. 시스템 콜은 함수처럼 보이지만 `ecall`로 커널에 들어가는 경계 넘기다.
3. FD는 프로세스별 표 `ofile[]`의 칸 번호이고, 새 FD는 비어 있는 가장 작은 번호다.
4. `read`의 동작은 FD 뒤에 무엇이 있느냐가 정한다. 프로그램은 그것을 모른다.
5. `fork`는 메모리를 복사하고 FD를 공유한다. `exec`은 메모리만 바꾸고 FD는 유지한다.
6. 리다이렉션과 파이프는 셸이 `fork`와 `exec` 사이에서 FD를 바꿔 끼워 만든다.
7. 동시에 도는 프로세스의 출력은 섞일 수 있다. 시스템 콜 한 번에는 비용이 있다.